

# CI/CD Quality Gates for OS Images

qemu-img checks | QEMU/KVM boot smoke tests | Release audits

*The project explained end to end, in simple terms*

**The project (as written on the resume):** Cut image regressions reaching production 85% as measured by release audits by adding CI/CD gates that ran qemu-img conversion, resizing, compression, and integrity checks and booted candidate images under QEMU/KVM for networking, storage, systemd, and kernel smoke tests.

## What is inside this document

- Section 1 - The simple summary (the big picture)
- Section 2 - Key words explained in plain English
- Section 3 - Architecture diagram
- Section 4 - The architecture diagram explained, piece by piece
- Section 5 - Every component explained in detail
- Section 6 - The complete workflow, step by step
- Section 7 - Which gate catches which kind of problem
- Section 8 - How the 85% was measured with release audits
- Section 9 - Gate failures and what they mean
- Section 10 - Safeguards and a 60-second explanation

**Note:** The resume line gives the result and the types of checks. This document explains how such gates are typically built. Exact tools (Jenkins, GitLab CI, GitHub Actions, and so on) and thresholds may differ in your real setup, and the numbers in the charts are illustrative examples - your real numbers come from your own release audits.

# 1. The Big Picture (Simple Summary)

## 1.1 The one-line version

I added automatic "checkpoints" to our release pipeline. Every new OS image first had its file checked (is it valid, not corrupted, can it be converted, resized, and compressed safely?) and then was actually started in a virtual machine to test the kernel, system services, network, and disks. Bad images were blocked before release, and our release audits showed 85% fewer broken images reaching production.

## 1.2 Explained with an everyday example

Think of a car factory.

- **Before:** cars left the factory after a quick look. Some had faulty brakes or a dead battery, and customers found the problems on the road.
- **After:** every car must pass **two inspection stations**. Station 1 checks the parts without starting the car (paperwork, body, tires). Station 2 starts the engine on a test track and checks steering, brakes, lights, and radio.
- **Any failure** sends the car back to the factory. Only cars that pass both stations reach customers.
- **Afterwards**, a quality team **audits** every car that had a problem on the road and asks: which station should have caught this? Then that station gets a new check.

In this project, the car is an **OS image** (a QCOW2 disk file), the inspection stations are **CI/CD gates**, the test track is a **QEMU/KVM virtual machine**, and the quality team's review is the **release audit**.

## 1.3 The resume line, phrase by phrase

| Phrase                                         | What it means in simple words                                                                                   |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Cut image regressions reaching production 85%  | Far fewer broken images got to real customers. Example: 27 problems in a year before, 4 after.                  |
| Regression                                     | Something that worked in the previous image but is broken in the new one.                                       |
| as measured by release audits                  | The number comes from formal reviews of each release, which count problems found in production.                 |
| adding CI/CD gates                             | Automatic pass/fail checkpoints in the release pipeline. An image cannot move forward until it passes.          |
| ran qemu-img conversion, resizing, compression | Tested that the image file can be safely turned into other formats, made bigger, and compressed without damage. |

| Phrase                                               | What it means in simple words                                                                                  |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| and integrity checks                                 | Checked the file is complete, uncorrupted, and exactly the one that was built.                                 |
| booted candidate images under QEMU/KVM               | Actually started each new image in a fast local virtual machine.                                               |
| networking, storage, systemd, and kernel smoke tests | Quick, essential tests: does the network work, do disks work, do system services start, is the kernel healthy? |

## 1.4 Why this matters

- **One bad image can break thousands of machines**, because every new VM is copied from it.
- **Fixing a problem before release is far cheaper** than fixing it after customers are affected.
- **Automatic gates never get tired** and never skip a check when a release is urgent.
- **Evidence is saved for every release**, so audits and investigations are fast and factual.
- **The gates get stronger over time**: every problem that slips through becomes a new test.

## 2. Key Words in Plain English

| Word               | Simple meaning                                                                                              | Example                                |
|--------------------|-------------------------------------------------------------------------------------------------------------|----------------------------------------|
| OS image           | A saved copy of a whole computer disk, ready to start.                                                      | ol9-2026.09.qcow2                      |
| Candidate image    | A new image that wants to be released but has not been approved yet.                                        | The build from this morning.           |
| Last good image    | The version currently in production and known to work. Used for comparison.                                 | ol9-2026.08.qcow2                      |
| Regression         | Something that used to work but is broken in the new version.                                               | Network stops working after an update. |
| Production         | The real environment used by customers.                                                                     | Customer VMs in OCI regions.           |
| CI/CD              | Continuous Integration / Continuous Delivery - automation that builds, tests, and releases on every change. | Jenkins, GitLab CI, GitHub Actions     |
| Gate               | An automatic pass/fail checkpoint. Fail = the image stops here.                                             | Gate 1: file checks                    |
| QCOW2              | A disk-image file format used by QEMU and accepted by Oracle Cloud.                                         | image.qcow2                            |
| raw / VMDK         | Other disk-image formats. raw is a plain byte-for-byte copy; VMDK is VMware's format.                       | image.raw                              |
| qemu-img           | Command-line tool to inspect, check, convert, resize, compress, and compare disk images.                    | qemu-img check                         |
| Integrity          | The file is complete, not corrupted, and not changed.                                                       | Checksum matches.                      |
| Checksum (SHA-256) | A unique fingerprint of a file. Any change, even one byte, changes it.                                      | sha256sum                              |
| Backing file       | A QCOW2 file can depend on another file. A release image must not.                                          | overlay -> base                        |
| Leaked clusters    | Space in a QCOW2 file that is allocated but not used - a sign of an unclean write.                          | Reported by qemu-img check.            |
| QEMU               | Software that emulates a computer so you can run a virtual machine.                                         | qemu-system-x86_64                     |
| KVM                | A Linux kernel feature that makes QEMU virtual machines run at near-native speed.                           | -enable-kvm                            |

| Word                      | Simple meaning                                                                                        | Example                        |
|---------------------------|-------------------------------------------------------------------------------------------------------|--------------------------------|
| Overlay (copy-on-write)   | A temporary layer on top of an image; all changes go into the layer, so the original stays untouched. | overlay.qcow2                  |
| OVMF                      | UEFI firmware for QEMU, used to test UEFI boot.                                                       | OVMF_CODE.fd                   |
| Serial console log        | Text output of the boot process, captured to a file.                                                  | serial.log                     |
| Smoke test                | A quick test of the most important functions. If smoke comes out, stop.                               | Does it boot? Can I log in?    |
| Kernel                    | The core of Linux that talks to hardware.                                                             | 5.15.0-xxx.el9uek              |
| Kernel panic / oops       | Serious kernel errors. A panic stops the system; an oops is a bug report that often leads to trouble. | "Kernel panic - not syncing"   |
| dmesg                     | The kernel's own message log.                                                                         | dmesg --level=err              |
| systemd                   | The program that starts and manages all services on modern Linux.                                     | systemctl                      |
| Unit / failed unit        | A service or task managed by systemd; failed means it did not start correctly.                        | sshd.service failed            |
| cloud-init / NoCloud seed | cloud-init sets up a new VM. A NoCloud seed is a small local disk giving it test settings.            | seed.iso                       |
| DHCP / MTU / DNS          | Automatic IP address / maximum packet size / name lookup.                                             | MTU 9000                       |
| virtio                    | Fast virtual devices for disks and network in VMs.                                                    | /dev/vda, eth0                 |
| JUnit report              | A standard test-result file that CI tools can display.                                                | results.xml                    |
| Release audit             | A formal review of what went wrong after a release.                                                   | Quarterly audit report         |
| Escape                    | A problem that passed all gates and reached production.                                               | A missed network bug.          |
| Flaky test                | A test that sometimes fails for no real reason.                                                       | Timing-sensitive network test. |

### 3. Architecture Diagram

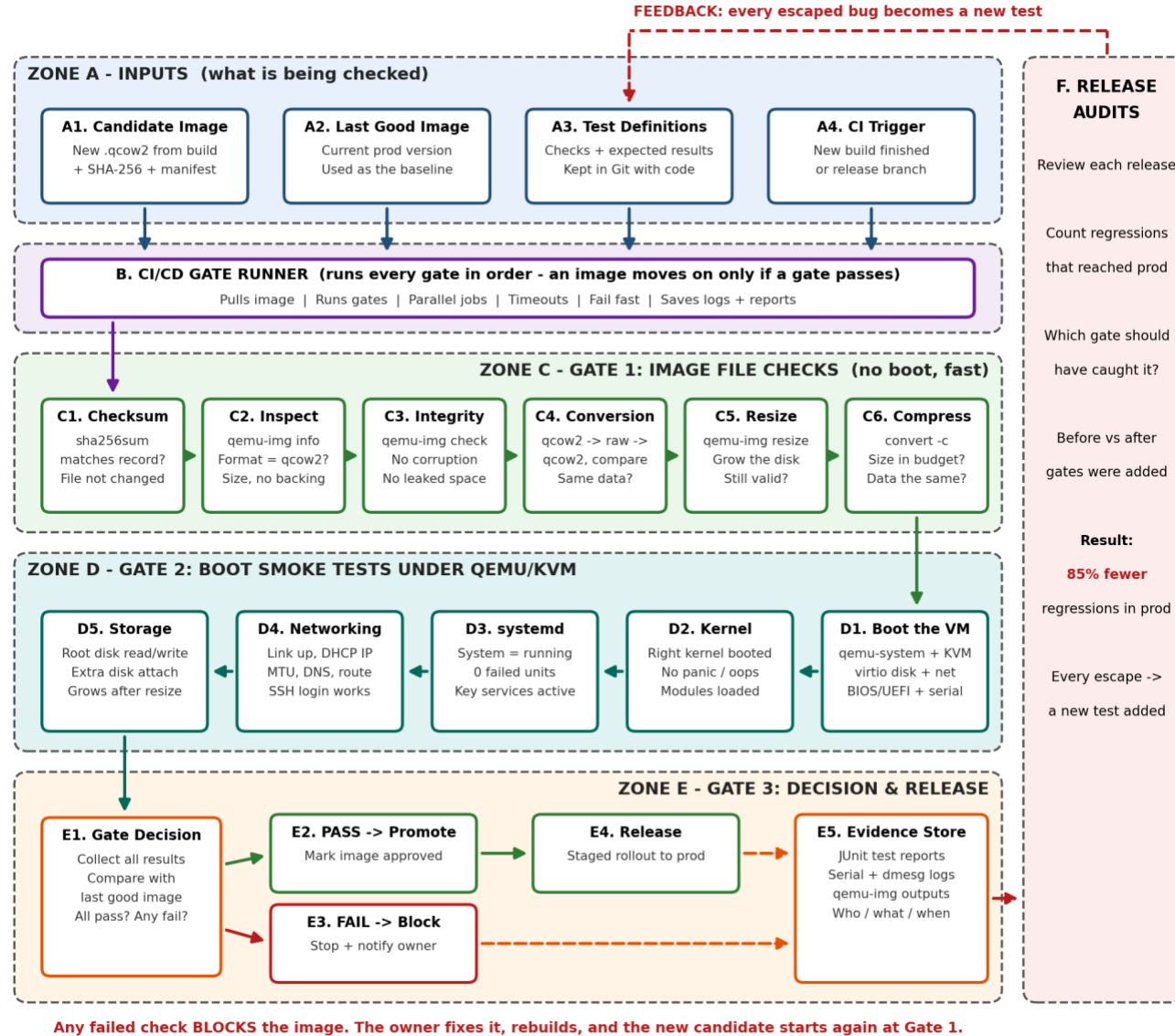


Figure 1 - End-to-end architecture of the CI/CD image gates. Labels (A1-F) match Sections 4 and 5.

## 4. The Architecture Diagram Explained, Piece by Piece

### 4.1 How to read the diagram

| Visual element               | What it means                                                                                      |
|------------------------------|----------------------------------------------------------------------------------------------------|
| Big dashed boxes (zones A-F) | Groups of parts with the same job: inputs, the gate runner, Gate 1, Gate 2, Gate 3, and audits.    |
| Small solid boxes            | Individual components. Labels match Section 5.                                                     |
| Blue (Zone A)                | Inputs - the image to check and what to check it against.                                          |
| Purple (Zone B)              | The CI/CD gate runner that controls all gates.                                                     |
| Green (Zone C)               | Gate 1 - file checks with qemu-img. No VM is started; very fast.                                   |
| Teal (Zone D)                | Gate 2 - the image is booted in a QEMU/KVM virtual machine and tested.                             |
| Orange (Zone E)              | Gate 3 - the pass/fail decision, release, and evidence.                                            |
| Red (Zone F)                 | Release audits - measuring how many problems still reach production (the 85% result).              |
| Solid arrows                 | The order in which checks happen.                                                                  |
| Dashed arrows                | Evidence and results being stored, and the feedback loop.                                          |
| Snake layout                 | C1 to C6 left to right, D1 to D5 right to left, then E left to right - only so arrows never cross. |

### 4.2 Zone A - Inputs

- **A1 Candidate Image:** the new QCOW2 file plus its checksum and manifest (list of contents).
- **A2 Last Good Image:** the version currently in production. The same tests run on it to tell real regressions apart from test problems.
- **A3 Test Definitions:** the list of checks and the expected results, stored in Git.
- **A4 CI Trigger:** a finished build or a release branch starts the gates.
- **Blue arrows:** all inputs go to the gate runner.

### 4.3 Zone B - CI/CD Gate Runner

One wide box because it controls every gate. It downloads the image, runs each gate in order, runs independent jobs in parallel, applies timeouts, stops on the first failure, and saves all logs and reports. The **purple arrow** starts Gate 1.

## 4.4 Zone C - Gate 1: image file checks (no boot)

- **C1 Checksum:** is this exactly the file that was built?
- **C2 Inspect:** is it really QCOW2, with the right size and no hidden dependency on another file?
- **C3 Integrity:** is the file's inside structure free of corruption and leaks?
- **C4 Conversion:** can it be converted to raw and back without any data changing?
- **C5 Resize:** can the disk be made bigger and stay valid?
- **C6 Compress:** does compression keep the data identical and the size within budget?
- **Green arrows:** each check runs only if the previous one passed. The **down arrow** from C6 takes the image to Gate 2.

## 4.5 Zone D - Gate 2: boot smoke tests under QEMU/KVM

- **D1 Boot the VM:** start the candidate in QEMU with KVM, using virtio disk and network, on BIOS and UEFI, and record the serial log.
- **D2 Kernel:** the right kernel is running and it reports no serious errors.
- **D3 systemd:** the system reached a healthy running state, no service failed, and key services are active.
- **D4 Networking:** the VM got an IP, has correct MTU, DNS and routing, and accepts SSH.
- **D5 Storage:** disks work, extra disks attach, and the root disk grows after resize.
- **Teal arrows:** tests go from most basic (boot) to more detailed. The **down arrow** from D5 goes to the decision.

## 4.6 Zone E - Gate 3: decision and release

- **E1 Gate Decision:** collects every result and compares against the last good image.
- **E2 PASS -> Promote:** the image is marked approved (green arrow).
- **E3 FAIL -> Block:** the image is stopped and the owner is notified (red arrow).
- **E4 Release:** approved images continue to the staged rollout to production.
- **E5 Evidence Store:** keeps test reports, logs, and outputs for every run, pass or fail (orange dashed arrows).

## 4.7 Zone F - Release audits and the feedback loop

After releases, audits count the regressions that still reached production and ask which gate should have caught each one. Comparing before and after the gates gives the **85% reduction**. The **red dashed arrow** from E5 brings the evidence to the audit, and the **top red dashed arrow** sends every escaped problem back to **A3** as a new test.

## 4.8 Every arrow in one table

| From     | To         | What travels along the arrow                                  |
|----------|------------|---------------------------------------------------------------|
| A1-A4    | B          | The candidate, the baseline, the tests, and the start signal. |
| B        | C1         | Start Gate 1.                                                 |
| C1 to C6 | Next check | The same image, trusted a bit more after each pass.           |
| C6       | D1         | An image whose file is proven healthy, ready to boot.         |
| D1 to D5 | Next test  | A running VM, tested one area at a time.                      |
| D5       | E1         | All Gate 1 and Gate 2 results.                                |
| E1       | E2 / E3    | PASS goes to promote; FAIL goes to block.                     |
| E2       | E4         | Approved image goes to release.                               |
| E3, E4   | E5         | Results, logs, and decisions are stored (dashed).             |
| E5       | F          | Evidence used by release audits (dashed).                     |
| F        | A3         | New tests for every escaped problem (feedback loop).          |

## 5. Every Component Explained in Detail

---

Each component has four parts: **In simple terms**, **Detailed breakdown**, **Why it matters**, and an **Everyday analogy**. Many also show a short example command (grey boxes). Labels match the diagram.

### A1 - Candidate Image

#### In simple terms

The new image that wants to be released.

#### Detailed breakdown

- **The QCOW2 file** produced by the build pipeline.
- **Checksum file** (SHA-256) written at build time.
- **Manifest**: OS version, kernel version, package list, build ID.
- **Read-only for the gates**: the tests never change this file; they work on copies or overlays.

#### Why it matters

The gates must test exactly what will ship.

**Everyday analogy**: A new car arriving at the inspection station with its paperwork.

### A2 - Last Good Image (baseline)

#### In simple terms

The image version currently in production and known to work.

#### Detailed breakdown

- **Same tests run on it** in the same environment.
- **If both fail**, the problem is probably the test or the environment, not the new image.
- **If only the candidate fails**, it is a real regression.
- **Performance comparison**: for example boot time must not be much slower than the baseline.

#### Why it matters

Separates real regressions from noisy tests, so the gates are trusted.

**Everyday analogy**: Comparing a new recipe with the old one that everybody liked.

### A3 - Test Definitions

#### In simple terms

The list of checks and expected answers, kept in Git next to the pipeline code.

### Detailed breakdown

- **What to run** (commands) and **what counts as pass** (expected output, exit code, time limit).
- **Per-image settings**, for example expected kernel version from the manifest.
- **Reviewed like code**: every new test goes through a pull request.
- **Grows over time**: each audit finding adds a test here.

### Why it matters

Tests are transparent, repeatable, and improve continuously.

**Everyday analogy:** The inspector's checklist.

## A4 - CI Trigger

### In simple terms

What starts the gates.

### Detailed breakdown

- **A finished image build** automatically starts the gates.
- **A release branch or tag** runs the full gate set.
- **Manual re-run** for investigations.
- **Required status**: the release step is configured so it cannot run unless the gates reported success.

### Why it matters

No image can skip the gates, even under time pressure.

**Everyday analogy:** The factory rule: every car goes through inspection, no exceptions.

## B - CI/CD Gate Runner

### In simple terms

The CI/CD system that runs every gate in order and makes the pass/fail call visible.

### Detailed breakdown

- **Runs on build agents** that support KVM (hardware virtualization enabled, /dev/kvm available).
- **Stages**: Gate 1, Gate 2, Gate 3 - each a stage in the pipeline.
- **Parallel jobs**: BIOS and UEFI boots, or x86 and Arm images, run at the same time.
- **Timeouts**: every job has a time limit so a hung VM cannot block the pipeline.
- **Fail fast**: the first real failure stops the run.
- **Clean workspace**: every run starts from an empty folder; temporary files and VMs are always removed.
- **Artifacts**: logs, reports, and outputs are uploaded to the evidence store.

- **Notifications:** results posted to the team chat or email.

### Why it matters

Makes the checks automatic, consistent, and impossible to skip.

**Everyday analogy:** The inspection line supervisor.

## C1 - Checksum Check

### In simple terms

Confirms the file is exactly the one that was built.

### Detailed breakdown

- **Recalculate** the SHA-256 of the downloaded file.
- **Compare** with the checksum recorded at build time.
- **Mismatch means** the file was damaged during copy or changed by someone - fail immediately.

### Example

```
sha256sum -c candidate.qcow2.sha256
```

### Why it matters

Prevents testing (and shipping) a damaged or wrong file.

**Everyday analogy:** Checking the seal on a package has not been broken.

## C2 - Inspect (qemu-img info)

### In simple terms

Reads the image's "label" to confirm it is the right kind of file with the right settings.

### Detailed breakdown

- **Format** must be qcow2.
- **Virtual size** must match the spec (for example 50 GB).
- **No backing file:** a release image must be standalone, not depend on another file.
- **Compatibility level** (compat 1.1) and cluster size as expected.
- **Corrupt flag** must be false.
- **JSON output** is parsed by the pipeline, not read by eye.

### Example

```
qemu-img info --output=json candidate.qcow2
```

### Why it matters

Catches wrong format or a missing base file that would fail on import.

**Everyday analogy:** Reading the label on a box before opening it.

## C3 - Integrity (qemu-img check)

### In simple terms

Scans the internal structure of the QCOW2 file for damage.

### Detailed breakdown

- **Checks internal tables** that map disk blocks to file locations.
- **Finds corruption** (broken references) and **leaked clusters** (wasted, orphaned space).
- **Exit code decides pass/fail:** 0 = clean; 2 = corruption; 3 = leaks; 1 = check could not run.
- **Any non-zero result blocks the image.**

### Example

```
qemu-img check candidate.qcow2 # expect: No errors were found
```

### Why it matters

A corrupted image may boot today and lose data tomorrow.

**Everyday analogy:** An X-ray of the car frame to find hidden cracks.

## C4 - Conversion Test

### In simple terms

Converts the image to another format and back, and proves no data changed.

### Detailed breakdown

- **qcow2 to raw:** a plain byte-for-byte copy.
- **raw back to qcow2.**
- **Compare** the original and round-trip copies with `qemu-img compare` - they must be identical.
- **Optional:** convert to other formats the platform or customers use (for example VMDK) and check them.
- **Why:** images are converted during import, export, and replication; conversion bugs break images silently.

### Example

```
qemu-img convert -O raw candidate.qcow2 work.raw
qemu-img convert -O qcow2 work.raw roundtrip.qcow2
qemu-img compare candidate.qcow2 roundtrip.qcow2
```

### Why it matters

Guarantees the image survives the format changes it will go through in real life.

**Everyday analogy:** Translating a letter to French and back to check nothing got lost.

## C5 - Resize Test

### In simple terms

Makes a copy of the disk bigger and checks it is still valid.

### Detailed breakdown

- **Work on a copy**, never the candidate.
- **qemu-img resize** grows the virtual disk (for example +10 GB).
- **qemu-img info and check** again: new size correct, no errors.
- **Linked to Gate 2:** the resized copy is booted, and the storage test (D5) checks that the partition and filesystem really grew.
- **Why:** customers almost always launch with a bigger disk than the image's default.

### Example

```
cp candidate.qcow2 resize.qcow2
qemu-img resize resize.qcow2 +10G
```

### Why it matters

A very common real-world action must not break the image.

**Everyday analogy:** Checking that an extendable table actually extends and stays stable.

## C6 - Compression Test

### In simple terms

Compresses the image and proves the data is unchanged and the size is acceptable.

### Detailed breakdown

- **qemu-img convert -c** creates a compressed copy.
- **qemu-img compare** proves the content is identical.
- **Size budget:** the compressed file must be below a limit (for example not more than 10% bigger than the last good image) - a big jump usually means leftover files.
- **Size trend** is saved as a metric.

### Example

```
qemu-img convert -c -O qcow2 candidate.qcow2 compressed.qcow2
qemu-img compare candidate.qcow2 compressed.qcow2
```

### Why it matters

Catches bloated images and compression problems that slow uploads and imports.

**Everyday analogy:** Vacuum-packing clothes and checking nothing got damaged.

## D1 - Boot the VM under QEMU/KVM

### In simple terms

Starts the candidate image as a real virtual machine on the CI machine.

### Detailed breakdown

- **Overlay first:** a temporary copy-on-write layer on top of the candidate, so the candidate is never changed.
- **KVM acceleration** so the VM runs at near-native speed.
- **virtio disk and network**, like in the cloud.
- **Two firmware types:** BIOS and UEFI (OVMF) - both must boot.
- **Extra blank data disk** attached for the storage test.
- **NoCloud seed disk** gives cloud-init a test SSH key and hostname.
- **Serial console** written to a log file; the boot is watched for a login prompt or cloud-init "finished" message.
- **Boot timeout**, for example 5 minutes. No login = fail.
- **SSH port forward** lets the test runner log in and run commands.

### Example

```
qemu-img create -f qcow2 -b candidate.qcow2 -F qcow2 overlay.qcow2
qemu-system-x86_64 -enable-kvm -cpu host -m 2048 -smp 2 -nographic \
  -drive file=overlay.qcow2,if=virtio -drive file=data.qcow2,if=virtio \
  -drive file=seed.iso,media=cdrom -serial file:serial.log \
  -netdev user,id=n0,hostfwd=tcp::2222-:22 -device virtio-net-pci,netdev=n0
```

### Why it matters

The only true proof an image works is to start it.

**Everyday analogy:** Turning the key and starting the engine on the test track.

## D2 - Kernel Smoke Tests

### In simple terms

Checks the core of the operating system is correct and healthy.

### Detailed breakdown

- **Right kernel:** `uname -r` matches the manifest.
- **No serious errors:** the serial log and `dmesg` contain no panic, oops, call trace, or BUG messages.
- **Drivers loaded:** virtio and other needed modules are present.
- **Boot options:** `/proc/cmdline` contains the required settings (for example `console=ttyS0`).
- **No taint** from unexpected modules.

### Example

```
uname -r
dmesg --level=emerg,alert,crit,err
grep -E 'Kernel panic|Oops|Call Trace|BUG:' serial.log
```

### Why it matters

Kernel problems are the most dangerous - they can crash every VM.

**Everyday analogy:** Checking the engine runs smoothly with no warning lights.

## D3 - systemd Smoke Tests

### In simple terms

Checks that all system services started properly.

### Detailed breakdown

- **Overall state:** systemctl is-system-running must say "running" ("degraded" means something failed).
- **No failed units:** systemctl --failed must be empty.
- **Key services active:** sshd, cloud-init, chronyd, firewalld, and the cloud agent.
- **Boot time:** systemd-analyze time must stay within budget compared with the last good image.
- **Clean first boot:** cloud-init reports status done without errors.

### Example

```
systemctl is-system-running --wait
systemctl --failed --no-legend
systemd-analyze time
cloud-init status --long
```

### Why it matters

A failed service can mean no login, no time sync, or no monitoring.

**Everyday analogy:** Checking every dashboard system - lights, radio, air-con - switched on.

## D4 - Networking Smoke Tests

### In simple terms

Checks the VM can talk to the network correctly.

### Detailed breakdown

- **Link up** on the primary interface.
- **DHCP address** received; default route present.
- **MTU** as expected.
- **DNS** name lookup works.
- **SSH login with key** works; **password login is refused**.
- **Firewall:** only expected ports open.
- **Interface naming** is stable (a common regression after kernel or udev changes).

### Example

```
ip -br addr; ip route show default
getent hosts example.com
ssh -p 2222 -o PasswordAuthentication=no testuser@localhost true
```

### Why it matters

A VM that cannot be reached is useless to the customer.

**Everyday analogy:** Checking the car's steering and that you can actually drive it out of the garage.

## D5 - Storage Smoke Tests

### In simple terms

Checks the disks work, extra disks attach, and resizing really works.

### Detailed breakdown

- **Root filesystem** is mounted read-write.
- **Write-read test:** write a file, sync, read it back, compare checksum.
- **Extra disk:** the blank data disk appears (for example /dev/vdb), can be formatted, mounted, and used.
- **Resize proof:** on the resized copy from C5, the root partition and filesystem grew to the new size.
- **fstab valid:** findmnt --verify shows no errors; UUIDs are correct.
- **No I/O errors** in dmesg.

### Example

```
findmnt / ; lsblk
dd if=/dev/urandom of=/tmp/t bs=1M count=64 && sync && sha256sum /tmp/t
findmnt --verify
```

### Why it matters

Storage bugs can lose customer data.

**Everyday analogy:** Checking the trunk opens, closes, and holds luggage safely.

## E1 - Gate Decision

### In simple terms

Collects every result and decides: pass or fail.

### Detailed breakdown

- **All required checks must pass.** One fail = block.
- **Baseline comparison:** failures that also happen on the last good image are flagged as environment or test problems and investigated separately.
- **Flaky test rule:** a failed test may be retried once; if it fails again, the image is blocked. Tests that are often flaky are fixed or quarantined with owner approval.
- **Thresholds:** for example boot time no more than 20% slower than baseline.
- **Clear summary:** which gate, which check, and a link to the log.

### Why it matters

One clear, automatic, consistent decision for every image.

**Everyday analogy:** The final inspector who signs or rejects the certificate.

## E2 - PASS: Promote

### In simple terms

Marks the image as approved.

### Detailed breakdown

- **Tag** the image as approved in the artifact store.
- **Record** the gate run ID in the manifest.
- **Unlock** the release step.

### Why it matters

Only approved images can move to production.

**Everyday analogy:** Stamping the certificate "PASSED".

## E3 - FAIL: Block

### In simple terms

Stops the image and tells the owner exactly what failed.

### Detailed breakdown

- **Pipeline status = failed;** the release step cannot run.
- **Notification** with the failing check and logs.
- **Image kept** (not deleted) for investigation.
- **Fix, rebuild,** and the new candidate starts again at Gate 1.

### Why it matters

Bad images never reach customers.

**Everyday analogy:** Sending the car back to the factory with a defect report.

## E4 - Release

### In simple terms

The approved image continues to the normal release process.

### Detailed breakdown

- **Publish** to the image catalog.
- **Staged rollout** to production (small first, then wider).
- **Monitoring** watches for problems after release.

### Why it matters

Gates reduce risk; staged rollout limits the damage of anything that still slips through.

**Everyday analogy:** Delivering cars to a few dealers first, then everywhere.

## E5 - Evidence Store

### In simple terms

Keeps proof of every test run.

### Detailed breakdown

- **JUnit XML reports** that CI and dashboards can display.
- **Logs:** serial console, dmesg, systemd journal, command outputs.
- **qemu-img outputs** (info, check, compare).
- **Metadata:** image version, who triggered it, when, and the decision.
- **Retention** long enough for audits (for example a year).

### Why it matters

Audits and investigations are based on facts, not memory.

**Everyday analogy:** The inspection records filed for every car.

## F - Release Audits

### In simple terms

Regular reviews that count how many regressions still reached production - the source of the 85% number.

### Detailed breakdown

- **Sources:** incidents, customer tickets, rollbacks, and post-release bug reports.
- **For each problem:** was it a regression? Which image version? Which gate should have caught it?
- **Count** regressions reaching production per period, before and after the gates.
- **Action:** every escape becomes a new test in A3.
- **Report** trends to leadership.

### Why it matters

Proves the gates work and shows where to improve them.

**Everyday analogy:** The quality team reviewing every car that broke down on the road.

## 6. The Complete Workflow, Step by Step

This follows one candidate image from the moment it arrives until it is released or blocked.

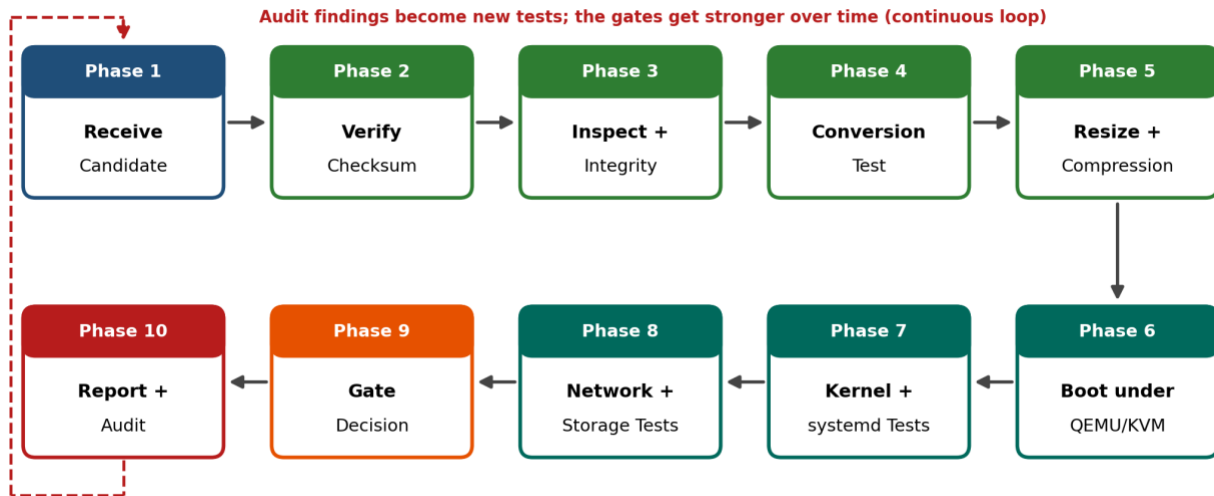


Figure 2 - The 10 phases. Blue = start, green = Gate 1, teal = Gate 2, orange = decision, red = report and audit.

### Phase 1 - Receive the candidate

| Step | What happens (detail)                                                | In simple words                  |
|------|----------------------------------------------------------------------|----------------------------------|
| 1.1  | The build finishes and the CI trigger starts the gate pipeline.      | The car arrives at inspection.   |
| 1.2  | A clean workspace is created on a KVM-capable agent.                 | A clean inspection bay.          |
| 1.3  | Download the candidate, checksum, manifest, and the last good image. | Bring the car and its paperwork. |

### Phase 2 - Verify the checksum

| Step | What happens (detail)                                  | In simple words       |
|------|--------------------------------------------------------|-----------------------|
| 2.1  | Recalculate SHA-256 and compare with the build record. | Check the seal.       |
| 2.2  | Mismatch: block immediately.                           | Broken seal = reject. |

### Phase 3 - Inspect and check integrity

| Step | What happens (detail)                                       | In simple words   |
|------|-------------------------------------------------------------|-------------------|
| 3.1  | qemu-img info: format, size, no backing file, compat level. | Read the label.   |
| 3.2  | qemu-img check: no corruption, no leaks (exit code 0).      | X-ray for cracks. |

## Phase 4 - Conversion test

| Step | What happens (detail)                                | In simple words               |
|------|------------------------------------------------------|-------------------------------|
| 4.1  | Convert qcow2 to raw and back to qcow2.              | Translate and translate back. |
| 4.2  | qemu-img compare the original and round-trip copies. | Nothing lost?                 |

## Phase 5 - Resize and compression tests

| Step | What happens (detail)                                                  | In simple words          |
|------|------------------------------------------------------------------------|--------------------------|
| 5.1  | Copy and grow the disk with qemu-img resize; check it again.           | Extend the table.        |
| 5.2  | Compress with qemu-img convert -c; compare content; check size budget. | Vacuum-pack and check.   |
| 5.3  | Gate 1 complete: the file is proven healthy.                           | Paperwork and body pass. |

## Phase 6 - Boot under QEMU/KVM

| Step | What happens (detail)                                                                            | In simple words                |
|------|--------------------------------------------------------------------------------------------------|--------------------------------|
| 6.1  | Create an overlay so the candidate is never modified.                                            | Use a test copy.               |
| 6.2  | Create the NoCloud seed (test SSH key) and a blank data disk.                                    | Prepare the test driver's key. |
| 6.3  | Start QEMU with KVM, virtio disk and network, BIOS and UEFI in parallel; capture the serial log. | Start the engine two ways.     |
| 6.4  | Wait for login / cloud-init done within the timeout; then connect over SSH.                      | Wait for the engine to settle. |

## Phase 7 - Kernel and systemd tests

| Step | What happens (detail)                                                                       | In simple words          |
|------|---------------------------------------------------------------------------------------------|--------------------------|
| 7.1  | Kernel version, no panic/oops/call trace, drivers loaded, boot options correct.             | Engine health check.     |
| 7.2  | System running, no failed units, key services active, boot time in budget, cloud-init done. | All systems switched on. |

## Phase 8 - Networking and storage tests

| Step | What happens (detail)                                                                             | In simple words              |
|------|---------------------------------------------------------------------------------------------------|------------------------------|
| 8.1  | Link, DHCP IP, route, MTU, DNS, SSH with key, password refused, firewall.                         | Can it drive and be reached? |
| 8.2  | Root read-write, write/read checksum, extra disk usable, resize grew the filesystem, fstab valid. | Does the trunk work?         |
| 8.3  | Shut down VMs, delete overlays and temporary files.                                               | Park and clean the bay.      |

## Phase 9 - Gate decision

| Step | What happens (detail)                                                         | In simple words      |
|------|-------------------------------------------------------------------------------|----------------------|
| 9.1  | Collect all results and compare with the last good image.                     | Final review.        |
| 9.2  | All pass: tag approved and unlock release.                                    | Stamp PASSED.        |
| 9.3  | Any fail: block, notify owner with logs; fix and rebuild restarts at Phase 1. | Back to the factory. |

## Phase 10 - Report and audit

| Step | What happens (detail)                                                 | In simple words        |
|------|-----------------------------------------------------------------------|------------------------|
| 10.1 | Upload JUnit reports, logs, and outputs to the evidence store.        | File the records.      |
| 10.2 | Approved images go through staged rollout and monitoring.             | Careful delivery.      |
| 10.3 | Release audits count production regressions and trace each to a gate. | Review road problems.  |
| 10.4 | Every escape becomes a new test in the test definitions.              | Improve the checklist. |

## 7. Which Gate Catches Which Kind of Problem

This table shows the kinds of regressions the gates were designed to stop.

| Type of regression                      | Real-world effect if released                 | Caught by                     |
|-----------------------------------------|-----------------------------------------------|-------------------------------|
| File corrupted during build or copy     | Import fails or random data errors            | C1 Checksum, C3 Integrity     |
| Wrong format or hidden backing file     | Image cannot be imported or launched          | C2 Inspect                    |
| Image breaks during format conversion   | Replication or export produces a broken image | C4 Conversion                 |
| Disk does not grow when launched bigger | Customer gets a small, full disk              | C5 Resize + D5 Storage        |
| Image suddenly much larger              | Slow uploads, imports, and launches           | C6 Compression                |
| Missing driver in initramfs             | VM does not boot                              | D1 Boot                       |
| UEFI or BIOS boot broken                | VMs on some machine types do not boot         | D1 Boot (both firmware types) |
| Wrong or crashing kernel                | Random crashes                                | D2 Kernel                     |
| Service fails to start                  | No login, no time sync, no monitoring         | D3 systemd                    |
| Network naming or DHCP broken           | VM unreachable                                | D4 Networking                 |
| SSH hardening broken or too strict      | Security risk, or customers locked out        | D4 Networking                 |
| fstab or mount errors                   | Boot hangs or data disk missing               | D5 Storage                    |
| Boot much slower                        | Slow scaling for customers                    | D3 systemd (boot time budget) |

## 8. How the 85% Was Measured with Release Audits

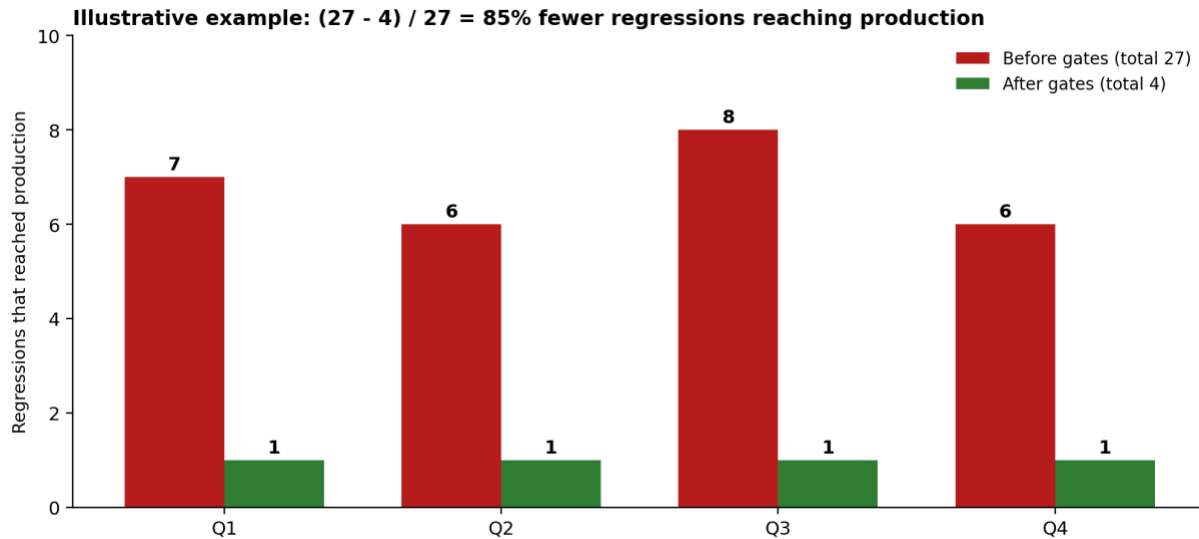


Figure 3 - Illustrative only. Real numbers come from your own release audits.

- **Define a regression clearly:** something that worked in the previous image and is broken in the new one, found in production.
- **Collect the facts:** incidents, support tickets, rollbacks, and bug reports linked to an image version.
- **Classify each one** in the audit: is it an image regression? Which release? Which gate should have caught it?
- **Compare equal periods:** for example four quarters before the gates versus four quarters after, with a similar number of releases.
- **Formula:** reduction = (before - after) / before. Example:  $(27 - 4) / 27 = 85\%$ .
- **Normalize if needed:** if the number of releases changed a lot, compare regressions per release instead of totals.
- **Close the loop:** each of the remaining escapes gets a new test, so the number keeps going down.

## 9. Gate Failures and What They Mean

| Failing check                    | What it usually means                                    | Typical fix                                |
|----------------------------------|----------------------------------------------------------|--------------------------------------------|
| Checksum mismatch                | File damaged during transfer, or wrong file              | Re-copy; check storage and transfer path   |
| qemu-img check: leaks            | Build tool did not close the image cleanly               | Fix cleanup in build; re-convert image     |
| qemu-img check: corruption       | Serious build or storage problem                         | Rebuild; investigate build host            |
| compare after conversion differs | Conversion bug or unstable image data                    | Update tools; investigate image content    |
| Compressed size over budget      | Leftover caches, logs, or big new packages               | Clean up in build; review package list     |
| No login before timeout          | Missing driver in initramfs, bootloader or fstab problem | Read serial log; fix initramfs or GRUB     |
| Boots on BIOS, not UEFI          | EFI partition or files missing                           | Fix EFI setup in build                     |
| Panic / oops in log              | Kernel bug or wrong kernel                               | Pin or change kernel version               |
| System state degraded            | A service failed to start                                | systemctl --failed and journal; fix config |
| No IP / wrong interface name     | Network config or udev rules changed                     | Fix network or naming config               |
| SSH refused                      | SSH hardening broke login or cloud-init did not add key  | Check sshd config and cloud-init logs      |
| Filesystem did not grow          | growpart or cloud-init resize missing                    | Add package or config back                 |
| Fails on candidate AND baseline  | Test or environment problem                              | Fix the test or CI agent, not the image    |

## 10. Safeguards and a 60-Second Explanation

---

### 10.1 Built-in safeguards

- **Gates are required:** the release step cannot run without a pass.
- **Candidate is never modified:** tests use copies and overlays.
- **Cheap checks first:** file checks in seconds, boot tests only on healthy files.
- **Both firmware types** tested every time.
- **Baseline comparison** separates real regressions from test noise.
- **Timeouts and cleanup** keep CI agents healthy.
- **Evidence kept** for every run.
- **Audits feed new tests**, so the gates keep getting stronger.

### 10.2 The 60-second explanation (for interviews)

"Broken OS images were occasionally reaching production. I added CI/CD gates that every candidate image had to pass. The first gate checked the file itself with qemu-img: checksum, info and check for corruption and leaks, a qcow2-to-raw-and-back conversion with compare, a resize test, and a compression test with a size budget. The second gate booted the image under QEMU/KVM on BIOS and UEFI, using an overlay so the candidate stayed untouched, and ran smoke tests: kernel health, systemd state and failed units, networking and SSH, and storage including disk growth. A decision stage compared results with the last good image, blocked any failure, and stored all evidence. Release audits showed an 85% drop in regressions reaching production, and every remaining escape became a new test."